

CrewAI & Multi-Agent Systems

From building blocks to AI teamwork — explained simply for students.

DR. ANILKUMAR VISHWANATH BRAHMANE

COMPUTER ENGINEERING DEPARTMENT, SANJIVANI COLLEGE OF ENGINEERING



What is a Multi-Agent System?

The Simple Idea

Instead of asking **one AI** to do everything, we create **multiple specialized AI agents** and make them work together — just like a team of people.

Each agent has its own **role, goal, tools, and instructions**.

Research Paper Example



📌 Multi-Agent System (MAS) = Multiple AI agents collaborating to accomplish a common goal.

What is CrewAI?

CrewAI is a Python framework for building AI systems where multiple agents collaborate on tasks and workflows. Think of **Crew = Team**.

Agent

An AI worker with a specific role

Task

The specific work assigned to an agent

Crew

The full team of agents working together

Process

How tasks are coordinated and executed



Core CrewAI Concepts: Agent, Task & Tools

Agent = AI Worker

Role

Who am I? e.g., "Senior AI Researcher"

Goal

What should I do? e.g., "Research Generative AI in healthcare"

Backstory

What expertise guides me?
Provides context and behavioral instructions

Tools

What can I use? e.g., Web Search, PDF Reader, Python, Database

Task = Work Assigned

1 Description

What should be done?

2 Expected Output

What should the result look like?

3 Agent

Who should do it?

❏ Output of one task becomes the input for the next task — like an assembly line!

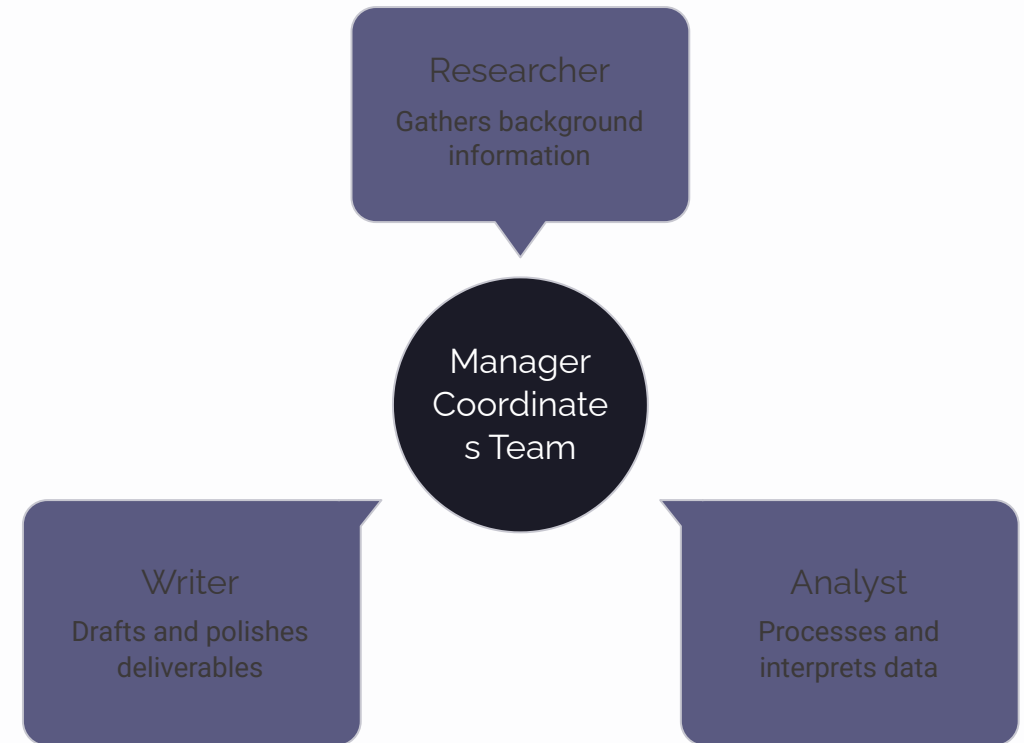
Sequential vs. Hierarchical Process

Sequential Process (Easy — Like Assembly Line)



Tasks run one after another in a fixed order. Best for beginners.

Hierarchical Process (Manager Coordinates Workers)



A Manager Agent coordinates workers dynamically. Best for complex, flexible workflows.

Sequential

Agent A → B → C → D (fixed order)

Hierarchical

Manager → Workers → Manager → Result

CrewAI Python Code — Step by Step

Install CrewAI first: `pip install crewai`

```
from crewai import Agent, Task, Crew, Process
```

```
# — STEP 1: Create an Agent (AI Worker) —
```

```
researcher = Agent(  
    role="AI Researcher",          # Who is this AI?  
    goal="Research Generative AI in education", # What should it do?  
    backstory=(  
        "You are an AI researcher who explains "  
        "technical topics in simple language."  
    ),                               # Context/instructions  
    verbose=True                   # Show execution details  
)
```

```
# — STEP 2: Create a Task (Work to be done) —
```

```
research_task = Task(  
    description="Research important applications of Generative AI in education.",  
    expected_output="A clear summary of major applications and their benefits.",  
    agent=researcher          # Who does this task?  
)
```

```
# — STEP 3: Create the Crew (Team) —
```

```
crew = Crew(  
    agents=[researcher],      # List of agents  
    tasks=[research_task],    # List of tasks  
    process=Process.sequential, # How tasks run  
    verbose=True  
)
```

```
# — STEP 4: Start the Crew —
```

```
result = crew.kickoff()      # Kick off the workflow!  
print(result)                # Display the result
```

Multi-Agent Python Example — 3 Agents Working Together

```
from crewai import Agent, Task, Crew, Process
```

```
# ——— AGENT 1: RESEARCHER ———
```

```
researcher = Agent(  
    role="AI Researcher",  
    goal="Research Generative AI in education",  
    backstory="You are an experienced AI researcher."  
)
```

```
# ——— AGENT 2: ANALYST ———
```

```
analyst = Agent(  
    role="Education Analyst",  
    goal="Analyze educational applications of Generative AI",  
    backstory="You analyze technology trends and their impact on education."  
)
```

```
# ——— AGENT 3: WRITER ———
```

```
writer = Agent(  
    role="Technical Writer",  
    goal="Write a clear report based on research and analysis",  
    backstory="You are an experienced technical writer."  
)
```

```
# ——— TASKS ———
```

```
research_task = Task(description="Research GenAI in higher education.",  
    expected_output="Summary of applications, benefits, challenges.",  
    agent=researcher)
```

```
analysis_task = Task(description="Analyze findings; identify opportunities & challenges.",  
    expected_output="A clear analytical summary.",  
    agent=analyst)
```

```
writing_task = Task(description="Prepare a professional report from research & analysis.",  
    expected_output="Well-structured report for engineering students.",  
    agent=writer)
```

```
# ——— CREW ———
```

```
crew = Crew(  
    agents=[researcher, analyst, writer],  
    tasks=[research_task, analysis_task, writing_task],  
    process=Process.sequential # Research → Analysis → Writing  
)
```

```
result = crew.kickoff()  
print(result)
```

Industry Applications of Multi-Agent Systems



Market Research

Researcher → Analyst → Competitor Agent → Writer → Reviewer produce a full business report automatically.



Software Development

Architect designs → Coder writes → Tester checks → Reviewer approves. Builds a REST API end-to-end.



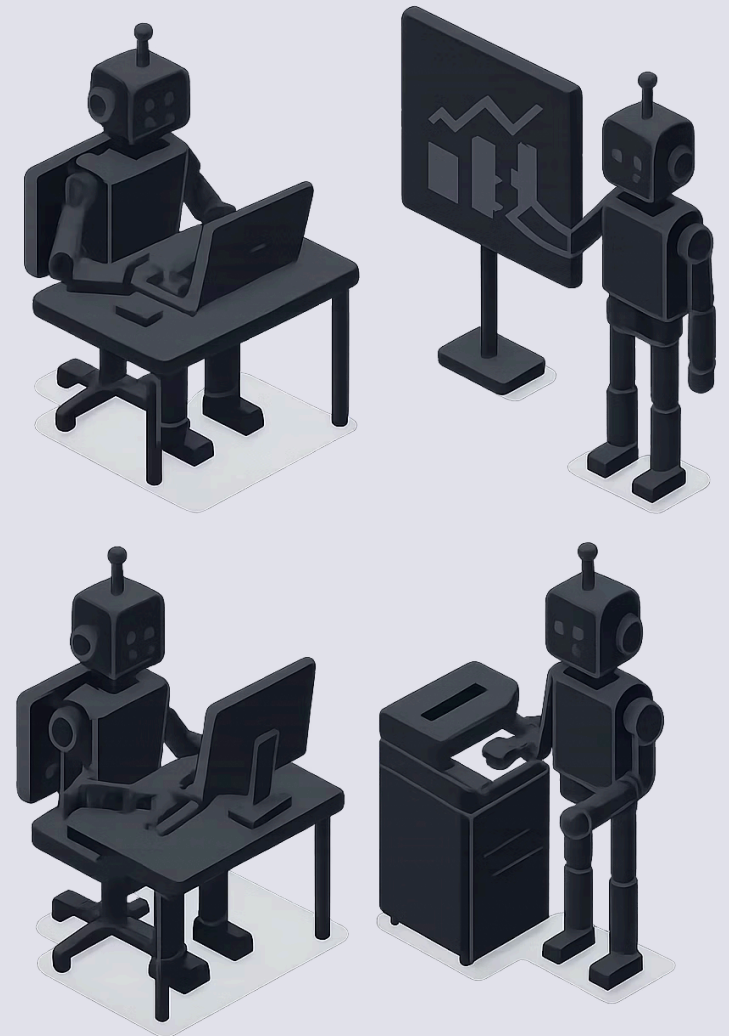
Healthcare (Decision Support)

Document Agent + Research Agent + Summary Agent assist clinicians. Human oversight is always required.



Education (AI Teaching Crew)

Researcher + Teacher Agent + Quiz Agent + Practical Agent produce a complete learning module with theory, examples, code, and MCQs.



Advanced Concepts & What Can Go Wrong

Advanced Concepts

→ Shared Memory

Agents share information through a common state so later agents can use earlier results.

→ Guardrails

Each agent has boundaries. e.g., Data Agent can READ but cannot DELETE records.

→ Human-in-the-Loop

For finance, healthcare, legal – a human reviews and approves before the agent acts.

→ Multi-Agent + RAG

Agents retrieve from a Vector DB (college documents, papers) for grounded answers.

What Can Go Wrong?

- **Error Propagation** – wrong info from Agent A corrupts Agent B and C
- Agents misunderstanding tasks
- Duplicate or conflicting work
- Infinite loops
- Hallucinated information
- Excessive API / token costs
- Security problems from over-privileged tools

⚠ More agents ≠ better system. Design carefully!

LangChain vs AutoGPT vs CrewAI — & The Golden Formulas

Comparison at a Glance

Feature	LangChain	AutoGPT	CrewAI
Core Idea	Building blocks	Autonomous agent	AI team
Agents	Single	One loop	Multiple specialized
Roles	Possible	Less central	Core concept
Collaboration	Limited	Limited	Central design
Memory Word	Build	Act	Collaborate

Three Golden Formulas

Single Agent

Agent = LLM + Goal +
Tools + State + Actions

Multi-Agent System

MAS = Agents + Roles +
Goals + Tasks +
Communication + Tools

CrewAI

CrewAI = Agents + Tasks + Tools + Processes + Crew

"One AI is a researcher, another is an analyst, another is a writer, another is a reviewer. They share information and work together. **This is a Multi-Agent System. CrewAI is the framework that makes it happen.**"